

Materials Society

AlgoAscent: Competitive Programming

Basic Project Introduction & Flow → IDE Setup → Full STL Usage & Understanding →
Time/Space Complexity Analysis → Recursion

Know C++ ?

☒ Select one

☐ Yes  33

☐ No  2

18:38 ✓

View votes

Know STL ?

☒ Select one

☐ Yes  18

☐ No  16

18:39 ✓

View votes

Know OOPS ?

☒ ☒ Select one or more

☐ Yes  7

☐ No  30

18:39 ✓

View votes

Number of Contests given on any coding platform

☒ Select one

☐ 0  21

☐ less than 5  8

☐ more than 5  7

18:40 ✓

View votes

Environment Setup: VS Code

A robust local environment is critical for speed. We avoid online compilers during practice to simulate contest conditions and enable debugging.

- **Step 1:** Install VS Code (Visual Studio Code).
- **Step 2:** Install a C++ Compiler (MinGW for Windows, Clang/GCC for Mac/Linux).
- **Step 3:** Add compiler 'bin' folder to System PATH environment variables.
- **Step 4:** Verify installation via terminal:
g++ --version

Every `cin/cout` call checks and syncs with `stdio`
C++ streams **stop syncing** with C streams
`cin / cout` become **much faster**

```
// Standard Template for CP
#include
using namespace std;

int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);

    // Your logic here
    return 0;
}
```

Use this boilerplate code to handle fast I/O.

```
C:\Users\aryan\CPP\contest\2025 april>rm aryan.exe
C:\Users\aryan\CPP\contest\2025 april>g++ algoAscent1.cpp -o aryan.exe
C:\Users\aryan\CPP\contest\2025 april>aryan.exe
Hello
C:\Users\aryan\CPP\contest\2025 april>
```

cmd
powershell

Critical VS Code Extensions

C/C++ by Microsoft

The core engine. It provides IntelliSense (code completion), debugging capabilities, and code browsing. It is non-negotiable for C++ development.

Key Feature: "Go to Definition" allows you to see how functions are implemented.

CPH (Competitive Programming Helper)

The ultimate time-saver. It integrates with your browser to automatically fetch test cases from Codeforces or LeetCode.

- Auto-runs your code against all sample inputs.
- Highlights 'WA' (Wrong Answer) or 'TLE' (Time Limit Exceeded).
- Eliminates manual copy-pasting of inputs.

| The Arena: Codeforces

Why Codeforces?

It is the gold standard for competitive programming, known for high-quality problems and a strict rating system.

Contest Structure

- **Div 4 & 3:** Beginner friendly. Good for learning implementation.
- **Div 2:** Intermediate. Introduces standard algorithms.
- **Div 1:** Expert. Advanced research-level topics.

Understanding Judgments

- **AC (Accepted):** Correct solution.
- **WA (Wrong Answer):** Failed a specific test case. Logic error.
- **TLE (Time Limit Exceeded):** Too slow. Check complexity.
- **RE (Runtime Error):** Crashed (e.g., dividing by zero, array out of bounds access).

Standard Template Library (STL)

vector, map, set, unordered_map,
unordered_set, queue, deque,
priority_queue

Do Not Reinvent the Wheel

The STL is a collection of C++ template classes providing common data structures and algorithms. In competitive programming, **implementation speed** is key. Instead of writing a sorting algorithm from scratch (which takes 10 minutes), use `sort()` (which takes less than 10 seconds).

```
// Example: Sorting an array
vector<int> v = {4, 2, 5, 1, 3};

// Without STL:
// Write 15 lines of Merge Sort code...

// With STL:
sort(v.begin(), v.end());
// v is now {1, 2, 3, 4, 5}
```

OOP Prerequisites: Templates

Why OOP matters here?

The STL (Standard Template Library) is built on the concept of **Generic Programming** (means you can declare data structure of any datatype).

- **Templates:** Allow writing a single function/class that works with *any* data type.
- **The 'T' in STL:** When you write `vector<int>`, 'int' replaces the generic template 'T'.
- **Encapsulation:** (function + properties) in a single class. Containers hide the messy memory management (pointers, allocation) behind simple functions like `push_back()`.

```
// How STL classes are roughly built
template <typename T>
class MyVector {
    T* arr;
    int capacity;
public:
    void push_back(T data) {
        // Logic to add data
    }
};

// Usage
MyVector<int> v1;
MyVector<string> v2;
```

Iterators & Functors

Iterators

Generalized pointers. They abstract *how* to move through a container.

- `begin()`: Points to first element.
- `end()`: Points **after** the last element.
- `*it`: Access value.
- `it++`: Move to next.

Functors

"Function Objects". Objects that can be treated like functions. Used for custom sorting.

Essential for Priority Queues and Sets to define custom order.

```
// Iterator Usage
vector<int> v = {10, 20};
auto it = v.begin();
cout << *it; // 10

// Functor (Comparator)
struct Comp {
    bool operator()(int a, int b) {
        return a > b; // Descending
    }
};
```

Search for “auto” keyword !!

| std::pair

The Utility Glue

A simple structure holding two values, which can be of different types.

- › **Usage:** Storing coordinates (x, y), graph edges (weight, node), or dictionary entries.
- › **Sorting:** By default, pairs sort by the first element. If equal, they check the second.

```
pair<int, string> p;  
p = {1, "Apple "}; // Assignment  
int id = p.first;  
string name = p.second;
```

Properties

Attribute	Detail
Header	#include <utility>
Comparison	Lexicographical (Compare 1st, then 2nd)
Nesting	pair<int,pair<int,int>> (for 3 items)

Search for tuples and their utility !!

| std::vector (Dynamic Array)

Description

Contiguous memory storage. It resizes automatically when elements are added.

Important Functions

- `push_back(x)`: Adds `x` to end.
- `pop_back()`: Removes last element.
- `reserve(n)`: Pre-allocate memory (Optimization).
- `back()`: Access last element.

Time Complexity

Access by index(<code>v[i]</code>)	$O(1)$
<code>v.front()</code> , <code>v.back()</code> , <code>v.pop_back()</code>	$O(1)$
Add at end → <code>v.push_back()</code>	amortized $O(1)$
<code>v.insert(pos, x)</code> , <code>v.erase(pos)</code>	$O(n)$

```
vector<int> v(5, 0); // Size 5, init 0
v.push_back(10);
for(int x : v) cout << x;
```

Containers inside Containers

Complex Data Modeling

In CP, simple lists aren't enough. You often need to nest structures.

Common Patterns

- **2D Matrix:** `vector<vector<int>>`. Dynamic rows and columns.
- **Adjacency List (Graphs):** `vector<vector<int>>`. Index `i` stores neighbors of node `i`.
- **Frequency of Lists:** `map<string, vector<int>>`. Key maps to a list of values.

```
// 2D Vector (Matrix)
vector<vector<int>> grid;
// Pushing a row {1, 2, 3}
grid.push_back({1, 2, 3});

// Accessing row 0, col 1
int val = grid[0][1]; // Returns 2

// Nested Map
map<string, vector<int>> studentGrades;
studentGrades["Alice"].push_back(95);
studentGrades["Alice"].push_back(88);
```

| std::deque (Double-Ended Queue)

Description

Indexable container that allows fast insertion and deletion at **both** the beginning and the end. Unlike vector, memory is not guaranteed to be contiguous.

Unique Functions

- `push_front(x)`: Add to start.
- `pop_front()`: Remove from start.
- `push_back(x) / pop_back()`: Standard end ops.

Time Complexity

Access <code>d[i]</code>	$O(n)$
<code>push_front(x)</code> , <code>pop_front()</code>	$O(1)$
<code>push_back(x) / pop_back()</code>	$O(1)$

```
deque<int> dq;  
dq.push_front(10);  
dq.push_back(20);  
// Memory is distributed  
chunks
```

| std::queue & std::stack

Note: No iterators. No random access. You can only touch the specific end.

Queue (FIFO)

First In, First Out. Used in BFS.

- `push(x)`: Add to back.
- `pop()`: Remove from front.
- `front()`: View first element.

```
q.push(1); q.pop();  
// All ops: O(1)
```

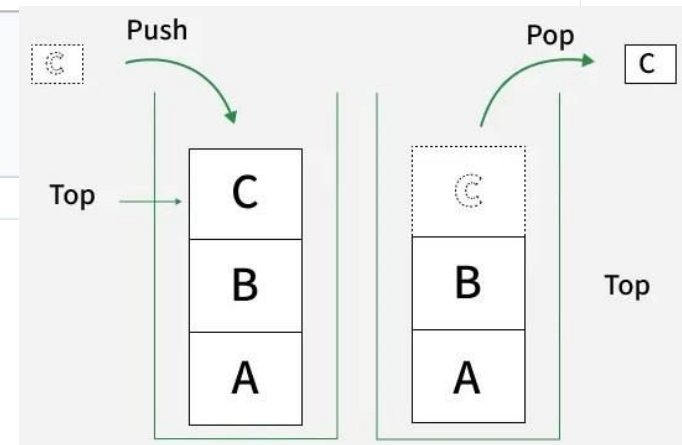


Stack (LIFO)

Last In, First Out. Used in DFS/Recursion.

- `push(x)`: Add to top.
- `pop()`: Remove from top.
- `top()`: View top element.

```
st.push(1); st.top();  
// All ops: O(1)
```



| std::priority_queue (Heap)

Description

A container adapter that provides constant time lookup of the largest (or smallest) element. Internally a Heap.

Variants

- **Max Heap (Default):** Largest element on top.
- **Min Heap:** Smallest element on top. Requires syntax: `priority_queue, greater>`.

Time Complexity

`top()` (Get Max/Min)

```
priority_queue<int> pq;  
pq.push(10); pq.push(30);  
cout << pq.top(); // 30  
pq.pop(); // Removes 30
```

✓ Method 1: Using `greater<>` (Min-Heap)

cpp

 Copy code

```
priority_queue<int, vector<int>, greater<int>> pq;
```

Set vs. Unordered Set

`std::set`

Structure: Red-Black Tree (Balanced BST).

Properties: Sorted, Unique elements.

Insert/Erase/Find	$O(\log N)$
-------------------	-------------

```
s.insert(5);  
// Iterating prints in sorted order
```

`std::unordered_set`

Structure: Hash Table.

Properties: Unsorted, Unique elements.

Insert/Erase/Find	$O(1)$ Avg
-------------------	------------

Worst Case	$O(N)$
------------	--------

```
us.insert(5);  
// Order is random
```

Map vs. Unordered Map

`std::map`

Structure: Red-Black Tree.

Key Feature: Keys are sorted. Good for finding ranges (lower_bound, upper_bound).

Access `m[key]` $O(\log N)$

```
m["Alice"] = 50;  
// Alice is stored before Bob
```

`std::unordered_map`

Structure: Hash Table.

Key Feature: Extremely fast lookups. Order is random.

Access `um[key]` $O(1)$ Avg

```
um["Alice"] = 50;  
// Fast hashing
```

The "Which one to use?" Cheat Sheet

Requirement	Use Container	Complexity
Dynamic List / Resizable Array	<code>vector</code>	$O(1)$ access
Insert at BOTH Front and Back	<code>deque</code>	$O(1)$
Key-Value Pair (Speed critical)	<code>unordered_map</code>	$O(1)$
Key-Value Pair (Sorted keys needed)	<code>map</code>	$O(\log N)$
Unique Elements Only	<code>unordered_set</code>	$O(1)$
Always need Largest/Smallest item	<code>priority_queue</code>	$O(\log N)$

Time Complexity (TC)

The 10^8 Rule

Most online judges (like Codeforces) execute roughly **10^8 operations per second**.

If the Time Limit is 1 second, your code must perform fewer than 100 million steps ($1e8$).

Input (N)	Allowable TC	Algorithm Type
$N \leq 10$	$O(N!)$	Permutations
$N \leq 10^3$	$O(N^2)$	Bubble/Insertion Sort
$N \leq 10^5$	$O(N \log N)$	Merge Sort, Maps
$N \leq 10^6$	$O(N)$	Linear Scan



But, Why do we see $5 * 10^5$ on codeforce ??

$1e8$ ops/sec → Hardware Limit

$5e5$ constraint → Algorithm design Perspective



Because real code does **much more** than `i++`

Due to - STL operations, Map / set /heap, Cache misses, Function calls, Memory accesses, I/O

Space Complexity (SC)

Memory Constraints

Usually 256MB. This is plenty, but not infinite.

Common Pitfalls

- **Large Arrays:** `int arr[10^8]` ~400MB. This causes Memory Limit Exceeded (MLE). Limit global arrays to $\sim 10^7$ and local arrays to $\sim 10^6$.
- **Recursion Depth:** Deep recursion (e.g., $N > 10^5$) fills the **Stack** memory, causing Stack Overflow.

Global vs Local

Stack (Local): Small (~8MB) of size $\sim 10^6$. Don't declare large arrays inside functions.

Heap/Data Segment (Global): Large. Always declare large arrays globally or use `std::vector` (which uses Heap).

Heap vs Stack Memory ?

Amortized Space Complexity (ASC)

Concept

Amortized analysis looks at the **average** cost per operation over a sequence, rather than the worst case of a single operation.

```
// Vector Push Back Analysis  
vector<int> v;  
for(int i=0; i  
    v.push_back(i);  
}
```

Scenario: When a vector is full, it doubles its capacity. Copying elements to the new space is **$O(N)$** .

However: Doubling happens rarely. Mathematically, across N insertions, the total cost is roughly $2N$. Thus, the average cost per insertion is **$O(1)$** .

Recursion: The Concept

Definition

A function that calls itself to solve a smaller instance of the same problem. It replaces complex loops with cleaner logic.

1. Base Case

Crucial. The condition that stops the recursion. Without this, the function calls itself infinitely, crashing the program (Stack Overflow).

2. Recursive Step

The logic where the problem size is reduced.
Example: To solve for N , we assume we know the answer for $N-1$.

The Stack

Every function call is added to the Call Stack. It remembers the state of variables at that specific level.

Recursion: Factorial Analysis

```
int factorial(int n) {  
    // 1. Base Case  
    if (n ≤ 1) return 1;  
  
    // 2. Recursive Step  
    int smaller_problem = factorial(n - 1);  
    return n * smaller_problem;  
}
```

Execution Flow for n = 3

- **Call fact(3):** Pauses, calls fact(2).
- **Call fact(2):** Pauses, calls fact(1).
- **Call fact(1):** Base case hit! Returns 1.
- **Resume fact(2):** Computes $2 * 1 = 2$. Returns 2.
- **Resume fact(3):** Computes $3 * 2 = 6$. Returns 6.

TC: $O(N)$ | SC: $O(N)$